Cell 1: Library Imports and WAV File Inspection

- **Purpose**: Imports essential libraries (numpy, soundfile, etc.) and loads an audio file to inspect its properties.
- **Explanation**: This cell reads a multichannel audio file and prints details like the number of channels, sample rate, and shape. This confirms that the audio is appropriate for multichannel processing (e.g., at least 2 channels are required for testing FastMNMF).

Cell 2: Basic Non-Negative Matrix Factorization (NMF) on Synthetic Data

- **Function**: nmf_basic_example()
- **Purpose**: Demonstrates basic NMF on synthetic data as an introduction.
- **Explanation**: This cell generates a random non-negative matrix V and decomposes it into two matrices, W and H, using multiplicative update rules. This shows how NMF can factorize data into components, an essential concept before applying it to audio.

Cell 3: Visualizing NMF Results

- **Function**: visualize_nmf(V, W, H, reconstructed)
- **Purpose**: Visualizes the original matrix V, factorized matrices W and H, and their reconstructed approximation.
- **Explanation**: This cell plots V, W, H, and the reconstructed W×H matrix to demonstrate how well NMF approximates the original data. Visualization helps in understanding the effectiveness of factorization.

Cell 4: Multichannel NMF on Audio Data

- **Function**: multichannel_nmf(audio_path, rank=5)
- **Purpose**: Applies NMF to each channel of a multichannel audio file.
- **Explanation**: This cell reads a multichannel audio file, extracts spectrograms for each channel, and then applies NMF to each channel to separate it into basis and activation components. For simplicity, only the first two channels are used initially (y = y[:, :2]). This prepares the data for FastMNMF by isolating distinct sources.

Cell 5: Running Multichannel NMF on Audio Data

- **Purpose**: Runs multichannel_nmf on an audio file path.
- **Explanation**: This cell applies the multichannel_nmf function to a specific audio file, using it to test the NMF process on real audio data. This cell initiates the audio processing workflow.

Cell 6: FastMNMF (Simplified Implementation)

- **Function**: fastmnmf(audio_path, rank=5, max_iterations=500)
- **Purpose**: Applies FastMNMF to separate sources in a multichannel audio file more efficiently.
- **Explanation**: This cell iteratively updates the matrices W and H using FastMNMF's optimized approach, which is better suited for handling multiple channels and converges faster than standard NMF. It again uses only the first two channels initially. FastMNMF's goal is to improve separation quality and efficiency for complex audio sources.

Cell 7: Run FastMNMF on Audio Data

- **Purpose**: Applies fastmnmf to the audio file path.
- **Explanation**: This cell uses the fastmnmf function on the selected audio file, executing the FastMNMF separation process. It takes the processed audio and attempts to decompose it efficiently using optimized updates.

Cell 8: Save Reconstructed Audio

- **Function**: save_reconstructed_audio(W, H, sr, output_folder="reconstructed")
- **Purpose**: Converts the separated spectrograms back into audio files and saves them.
- **Explanation**: This function reconstructs the audio from the factorized matrices W and H and saves the output as a .wav file. This allows you to listen to the separated audio sources to assess the quality of the separation.

Cell 9: Saving the Reconstructed Audio from FastMNMF

- **Purpose**: Calls save_reconstructed_audio with the outputs from FastMNMF.
- **Explanation**: This cell saves the audio results produced by FastMNMF, creating separate audio files for the reconstructed sources, allowing you to analyze the success of the source separation.

Summary

This notebook is structured to start with basic NMF on synthetic data, then extends to multichannel NMF on real audio files, and finally applies FastMNMF for more efficient separation. Each step builds on the previous one to deepen understanding and enhance the separation approach:

1. **Synthetic NMF**: Understands basic NMF with simple data.
2. **Multichannel NMF**: Applies NMF to actual audio data on each channel.
3. **FastMNMF**: Optimizes the NMF process for multichannel audio with improved convergence.
4. **Audio Reconstruction**: Saves and listens to the separated audio for quality assessment.